Final Project: Food Inspections

Chicago & Dallas Food Establishment Inspections Pipeline

Submitted by: Team 7 (Mehak, Komal, Gaurav)
Course: DAMG7370 Designing Data Architecture for Business Intelligence
Northeastern University
Spring 2026

Git Link: https://github.com/GauravBohra2001/Food-Establishment-Inspections-End-to-End-ETL-Pipeline.git

Part 1: Data Profiling and Source Analysis

1.1 Project Overview

This project analyzes food establishment inspection data from two cities: Chicago and Dallas, with the goal of uncovering public health insights and improving transparency. The pipeline follows the Medallion Architecture (Bronze → Silver → Gold) implemented on Databricks using Delta Tables, with data quality validation through custom DQX rules.

1.2 Source Datasets

Two datasets were ingested with significantly different schemas:

Attribute	Chicago	Dallas
Source Format	CSV	CSV
Row Count (Raw)	308,343	78,984
Inspection Identifier	Inspection_ID (numeric)	Derived via MD5 hash
Violations Structure	Single pipe-delimited text field	25 wide columns (Violation_Description_-_N etc.)
Violation Scoring	Derived from Results (Pass=90, Fail=70, etc.)	Numeric score (0-100)
Restaurant Identifier	Derived via MD5 hash (DBA_name + facility_type + address + zip)	Derived via MD5 hash (name + address + zip)
Risk Category	Embedded in Risk field (e.g., Risk 1 (High))	Not provided, defaulted to 'Unknown'
Facility Type	Provided directly	Derived using keyword matching on DBA name
State	Provided (IL)	Derived (TX)
Address	Single Address field	Split across Street_Number, Street_Name, Street_Direction, Street_Type, Street_Unit
Lat/Long	Separate columns	Embedded in Lat_Long_Location string

1.3 Schema Differences and Harmonization Strategy

Because both cities differ significantly in structure, a harmonization strategy was designed during the Bronze-to-Silver transformation:

- Chicago violations were stored as a single pipe-delimited string and parsed using regex to extract violation code, description, and inspector comments.
- Dallas violations were stored in up to 25 wide columns per inspection row. These were unpivoted into one row per violation using a slot-based union pattern.
- Both datasets were standardized to a common SILVER_COLS schema: one row per violation per inspection.
- Violation IDs were generated via MD5 hash of violation_code, violation_description and violation_detail columns to uniquely identify each violation type.
- Restaurant IDs were generated via MD5 hash of DBA_Name, facility_type, address and zip_code columns to create a consistent entity key across both cities.
- Inspection IDs for Dallas were generated via MD5 of DBA_Name, inspection_date, address and zip_code columns.

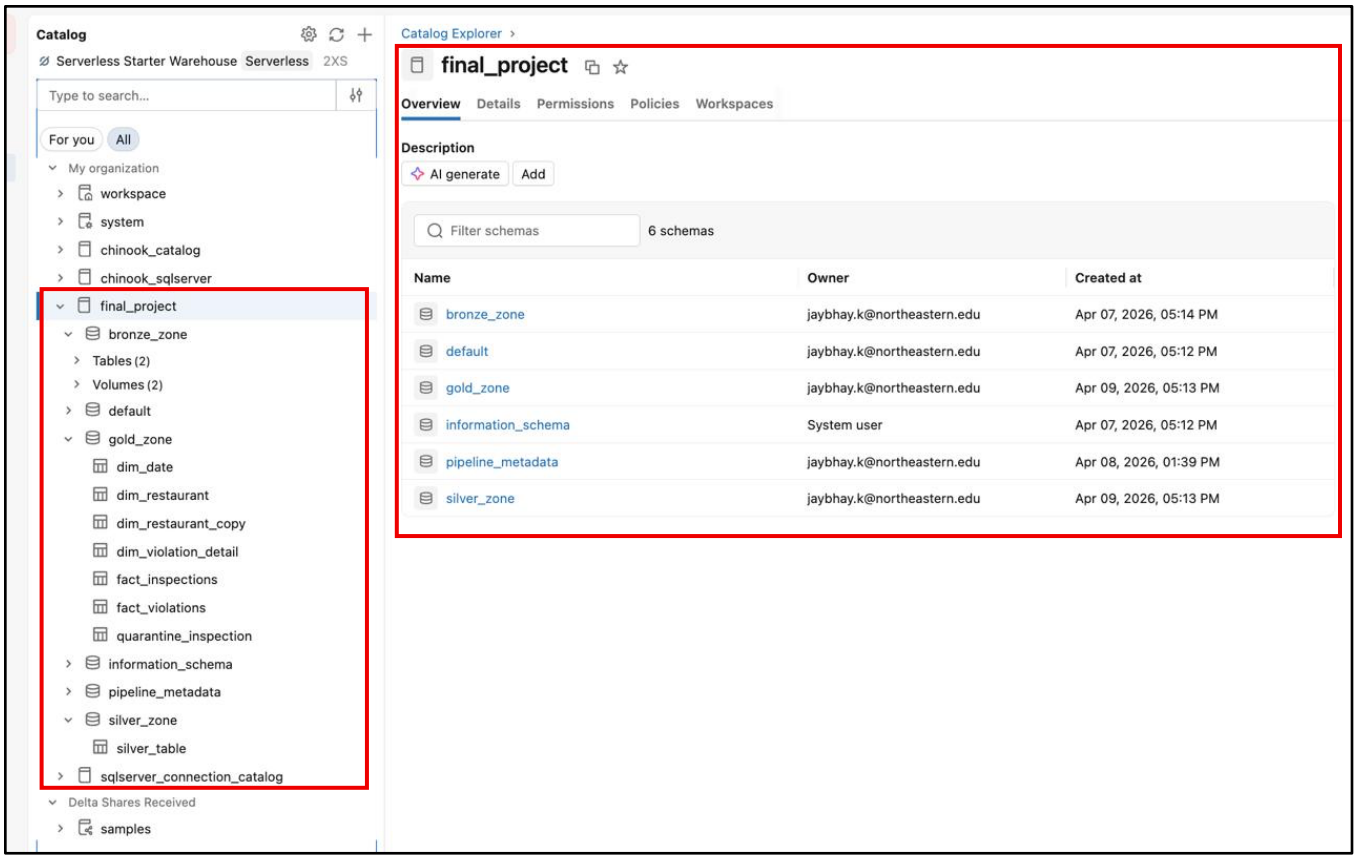
1.4 Data Profiling Findings

Chicago

- 308,343 raw rows ingested into bronze_chicago.
- Violations stored as unstructured text required regex-based parsing to extract violation code and description.
- Inspection results (Pass, Fail, Pass w/ Conditions, No Entry) were mapped to numeric scores for consistency with Dallas.
- Chicago does not record violation points natively. The columns “is_critical_flag” was derived based on inspection result and violation content.

Dallas

- 78,984 raw rows ingested into bronze_dallas.
- Wide violation schema with up to 25 violation slots per inspection.
- Facility type was not explicitly provided and was derived using keyword-matching logic on DBA_Name (example: 'starbucks' -> Cafe, 'taqueria' -> Restaurant).
- Risk category was not available and set to 'Unknown' for all Dallas records.
- Latitude/longitude were embedded in a single string column and extracted via regex.
- DQX found 1,300 bad rows in Dallas: 1,186 with invalid zip codes, 75 with null DBA names, and 39 with out-of-range violation scores.


Catalog
Serverless Starter Warehouse Serverless 2XS
Type to search...
For you
All
My organization
workspace
system
chinook_catalog
chinook_sqlserver
final_project
bronze_zone
Tables (2)
Volumes (2)
default
gold_zone
dim_date
dim_restaurant
dim_restaurant_copy
dim_violation_detail
fact_inspections
fact_violations
quarantine_inspection
information_schema
pipeline_metadata
silver_zone
silver_table
sqlserver_connection_catalog
Delta Shares Received
samples
Catalog Explorer
final_project
Overview
Details
Permissions
Policies
Workspaces
Description
AI generate
Add
Filter schemas
6 schemas
Name
Owner
Created at
bronze_zone
jaybhay.k@northeastern.edu
Apr 07, 2026, 05:14 PM
default
jaybhay.k@northeastern.edu
Apr 07, 2026, 05:12 PM
gold_zone
jaybhay.k@northeastern.edu
Apr 09, 2026, 05:13 PM
information_schema
System user
Apr 07, 2026, 05:12 PM
pipeline_metadata
jaybhay.k@northeastern.edu
Apr 08, 2026, 01:39 PM
silver_zone
jaybhay.k@northeastern.edu
Apr 09, 2026, 05:13 PM

172 print("\nRaw load completed...")
See performance (15)
active_tables_df: pyspark.sql.connect.dataframe.DataFrame = [table_name: string]
staging_df: pyspark.sql.connect.dataframe.DataFrame = [Inspection ID: string, DBA Name: string ... 15 more fields]
target_df: pyspark.sql.connect.dataframe.DataFrame = [Inspection_ID: string, DBA_Name: string ... 18 more fields]
df_bronze: pyspark.sql.connect.dataframe.DataFrame = [Inspection_ID: string, DBA_Name: string ... 19 more fields]
Active files to process: ['Dallas', 'Chicago']
Processing Dallas at 2026-04-12 17:53:23.410857
Dallas - 78,984 rows written to staging
Written to: /Volumes/Final_Project/Bronze_Zone/vol_stg_files/2026/04/12/dallas_restaurant_and_food_establishment_inspections_20260412_175330.parquet
Dallas | SUCCESS | Source: 78984 | Target: 78984 | Batch ID: BATCH_20260412_175330
Written to: Final_Project.Bronze_Zone.bronze_dallas
Processing Chicago at 2026-04-12 17:53:54.084806
Chicago - 308,343 rows written to staging
Written to: /Volumes/Final_Project/Bronze_Zone/vol_stg_files/2026/04/12/chicago_restaurant_and_food_establishment_inspections_20260412_175359.parquet
Chicago | SUCCESS | Source: 308343 | Target: 308343 | Batch ID: BATCH_20260412_175359
Written to: Final_Project.Bronze_Zone.bronze_chicago

```
34
See performance (3)
chi_good:  pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 24 more fields]
chi_bad:  pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 25 more fields]
Chicago — good: 1,001,903  bad: 49

Bad rows by rule:
+----------------------------------+-----+
|_dq_error                         |count|
+----------------------------------+-----+
|zip_code is null or invalid format|49   |
+----------------------------------+-----+
```

```
dal_good:  pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 24 more fields]
dal_bad:  pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 25 more fields]
Dallas  — good: 397,106  bad: 1,300

Bad rows by rule:
+----------------------------------+-----+
|_dq_error                         |count|
+----------------------------------+-----+
|zip_code is null or invalid format|1186 |
|dba_name is null/empty            |75   |
|violation_score out of range 0-100|39   |
+----------------------------------+-----+
```

05:44 PM (16s) 11 Python

```python
# — CELL 8 : Business rules — Dallas ————————————————————
from pyspark.sql.window import Window

w = Window.partitionBy("inspection_id")

dal_good = (
    dal_good
    # Rule 1: PASS cannot have critical violation
    .filter(~((F.col("inspection_result") == "PASS") &
              (F.col("is_critical_flag")  == "Y")))
    # Rule 2: score >= 90 cannot have more than 3 violations
    .withColumn("_viol_cnt", F.count("*").over(w))
    .filter(~((F.col("violation_score") >= 90) &
              (F.col("_viol_cnt") > 3)))
    .drop("_viol_cnt")
)

print(f"Dallas final : {dal_good.count():,} rows")
```

```
See performance (1)
dal_good:  pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 24 more fields]
Dallas final : 240,800 rows
```

```
chi_good: pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 24 more fields]
dal_good: pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 24 more fields]
Chicago after dedup : 939,632
Dallas  after dedup : 240,211
```

05:46 PM (35s) 13 Python

```python
# — CELL 10 : Union → single silver table ——————————————
silver = (
    chi_good
    .unionByName(dal_good)
    .withColumn("_silver_loaded_at", F.current_timestamp())
)

print(f"Unified silver : {silver.count():,} rows")
print("\nSource city breakdown:")
silver.groupBy("source_city").agg(
    F.count("*").alias("violation_rows"),
    F.countDistinct("inspection_id","inspection_date","restaurant_id","violation_score").alias("unique_inspections")
).show()
print("\nSilver schema:")
silver.printSchema()
```

See performance (2)

```
silver: pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 25 more fields]
Unified silver : 1,179,843 rows

Source city breakdown:
+-----------+--------------+------------------+
|source_city|violation_rows|unique_inspections|
+-----------+--------------+------------------+
|     DALLAS|        240211|             57738|
|    CHICAGO|        939632|            221740|
+-----------+--------------+------------------+


Silver schema:
root
```

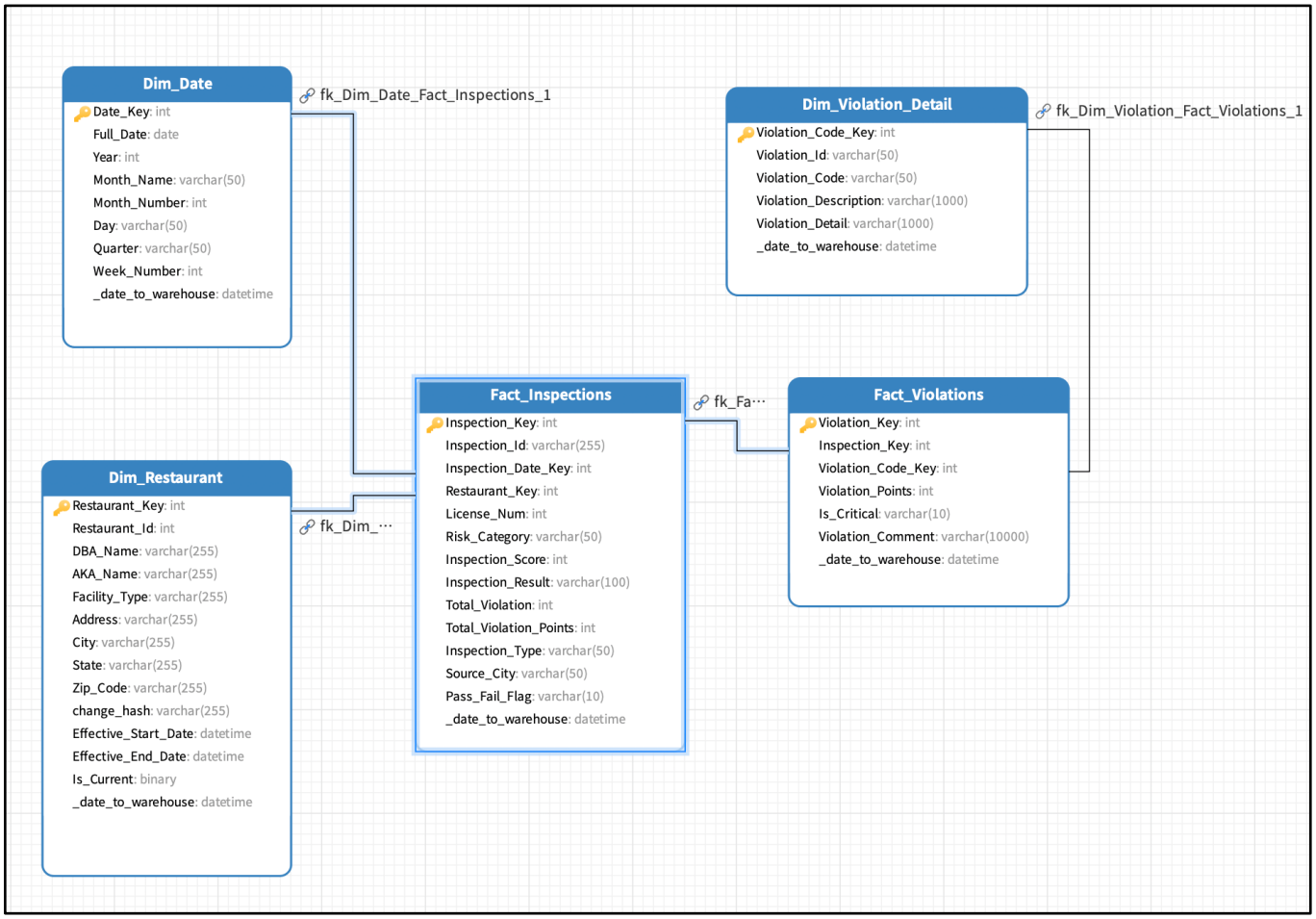
```
st: pyspark.sql.connect.dataframe.DataFrame = [inspection_id: string, source_city: string ... 25 more fields]
silver.silver_table     : 1,179,843 rows  27 cols
Inspections with 0 viols: 0
PASS + critical viols   : 0
Dallas score > 100      : 0
Dallas score>=90 >3viols: 0
Invalid zip codes       : 0

ALL CHECKS PASSED
```

```
============================================================
SILVER LAYER SUMMARY
============================================================
Table : food_inspections.silver.silver_table
Rows  : 1,179,843   Cols: 27

Rows by city:
+-----------+--------------+------------------+
|source_city|violation_rows|unique_inspections|
+-----------+--------------+------------------+
|     DALLAS|        240211|             57738|
|    CHICAGO|        939632|            221740|
+-----------+--------------+------------------+

Score distribution:
+-----------+------------------+---------------+------+
|source_city| inspection_result|violation_score| count|
+-----------+------------------+---------------+------+
|    CHICAGO|         NOT READY|           NULL|   426|
|    CHICAGO|   OUT OF BUSINESS|           NULL|   139|
```

Part 2: Dimensional Model Design

2.1 Modeling Approach

The dimensional model is a star schema and was designed to support analytical queries across inspection results, violation details, restaurant characteristics, and time. The model consists of two fact tables and three dimension tables, stored in the Gold Zone.



2.2 Dimensional Model Overview

Table	Type	Grain
dim_date	Dimension	One row per unique inspection date
dim_restaurant	Slowly Changing Dimension (Type 2)	One row per version of a restaurant entity
dim_violation_detail	Dimension	One row per unique violation code/description combination
fact_inspections	Fact	One row per inspection event
fact_violations	Fact	One row per violation per inspection

2.3 Table Definitions

dim_date

A standard date dimension built from all distinct inspection dates found in the Silver table. Attributes include year, month name, month number, quarter, day of week, and week number.

Separating date attributes into a dedicated dimension enables time-based slicing and dicing without storing redundant calendar data in the fact table. The date_key is formatted as YYYYMMDD (integer) for fast joining.

dim_violation_detail

Stores the catalogue of all unique violations observed across both cities. Each row represents a distinct violation, identified by a violation_id (MD5 of code + description + detail). Attributes include violation_code, violation_description, and violation_detail.

Violations are descriptive, relatively static reference data. Separating them from the fact table avoids repeating long description strings per inspection row, reduces storage, and makes violation filtering and lookup efficient. Dallas and Chicago violation codes are maintained separately but are stored in the same dimension table, accommodating both city schemas.

dim_restaurant (SCD Type 2)

Tracks restaurant entities with full history using Slowly Changing Dimension Type 2 logic. Each row represents one version of a restaurant, identified by restaurant_id (MD5 hash). Tracked attributes include: DBA_Name, AKA_Name, facility_type, address, city, state, and zip_code.

The change_hash (MD5 of AKA_Name) is used to detect changes. When a change is detected, the current record is expired (is_current = false, effective_end_date = current timestamp) and a new record is inserted (is_current = true, effective_end_date = 9999-12-31).

Since Restaurants change names, addresses, and facility types over time, SCD Type 2 preserves this history, allowing historical inspections to be analyzed against the restaurant's state at the time of inspection. The MERGE pattern using a NULL-key branch ensures clean separation of update and insert operations in a single Delta MERGE statement.

fact_inspections

Central fact table capturing one row per inspection event. It joins to dim_restaurant via restaurant_key (FK), and to dim_date via inspection_date_key (FK). Key metrics include: inspection_score, total_violation, total_violation_points, pass_fail_flag and inspection_result.

The inspection event is the primary analytical subject. Separating it from violation-level detail allows efficient querying at the inspection level without inflating row counts with per-violation data.

fact_violations

Fact table capturing one row per violation per inspection. Joins to fact_inspections via inspection_key (FK) and to dim_violation_detail via violation_code_key (FK). Additional attributes include violation_points, is_critical flag, and violation_comment (inspector notes).

The many-to-many relationship between inspections and violations cannot be modeled as a simple dimension. A separate violation fact table allows querying at the violation level, e.g., which violations appear most frequently, which are most critical, while maintaining a clean fact_inspections table at inspection grain since the requirements had analysis on different grains for violations and inspections.

2.4 Foreign Key Relationships

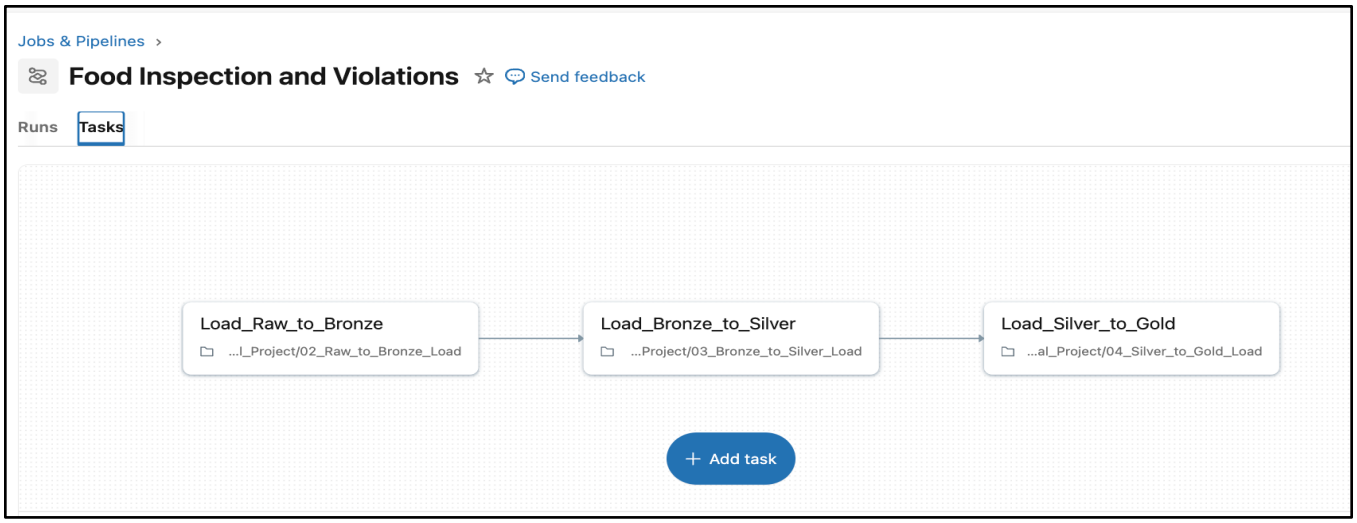
Relationship	From Table	FK Column	To Table	PK Column
Inspection -> Date	fact_inspections	inspection_date_key	dim_date	date_key
Inspection -> Restaurant	fact_inspections	restaurant_key	dim_restaurant	restaurant_key
Violation -> Inspection	fact_violations	inspection_key	fact_inspections	inspection_key
Violation -> Violation Detail	fact_violations	violation_code_key	dim_violation_detail	violation_code_key

Part 3: Loading the Dimensional Model (Silver to Gold)

3.1 Pipeline Architecture

The pipeline was orchestrated as a Databricks Job with three sequential tasks:

Task	Notebook	Description
Task 1	02_Raw_to_Bronze_Load	Reads CSVs from Raw Volume, writes Parquet to Staging Volume, loads Delta tables (bronze_dallas, bronze_chicago). Logs metrics to child metadata table.
Task 2	03_Bronze_to_Silver_Load	Transforms both bronze tables into a unified Silver schema, applies DQX rules, deduplicates violations, and writes to silver_table.
Task 3	04_Silver_to_Gold_Load	Reads silver_table and loads Gold Zone dimensional model: dim_date, dim_violation_detail, dim_restaurant (SCD2), fact_inspections, fact_violations.



3.2 Raw to Bronze (Notebook 02)

The Bronze load reads the active sources from a parent metadata table and processes each city independently. Key implementation details:

- Reads raw CSVs from the landing Volume using configurable path mappings (LANDING_FILE_MAP).
- Writes Parquet files to a Staging Volume with a timestamped path (YYYY/MM/DD/filename_YYYYMMDD_HHMMSS.parquet) for auditing and reprocessing.
- Row-count reconciliation: source count vs. target count is compared and status (SUCCESS/FAILED/SKIPPED) is logged to a child metrics metadata table.
- All column names are normalized (spaces replaced with underscores) before loading to Delta.
- Metadata columns added: _batch_id, _source_city, _ingested_at, _bronze_loaded_at.
- Dallas loaded: 78,984 rows to bronze_dallas. Chicago loaded: 308,343 rows to bronze_chicago.

```
Active files to process: ['Dallas', 'Chicago']


Processing Dallas at 2026-04-12 17:53:23.410857
Dallas - 78,984 rows written to staging
Written to: /Volumes/Final_Project/Bronze_Zone/vol_stg_files/2026/04/12/dallas_restaurant_and_food_establishment_inspections_20260412_175330.parquet
Dallas | SUCCESS | Source: 78984 | Target: 78984 | Batch ID: BATCH_20260412_175330

Written to: Final_Project.Bronze_Zone.bronze_dallas

--------------------------------------------------------------------------------

--------------------------------------------------------------------------------


Processing Chicago at 2026-04-12 17:53:54.084806
Chicago - 308,343 rows written to staging
Written to: /Volumes/Final_Project/Bronze_Zone/vol_stg_files/2026/04/12/chicago_restaurant_and_food_establishment_inspections_20260412_175359.parque
t
Chicago | SUCCESS | Source: 308343 | Target: 308343 | Batch ID: BATCH_20260412_175359

Written to: Final_Project.Bronze_Zone.bronze_chicago
```

3.3 Bronze to Silver (Notebook 03)

This notebook is the most complex transformation step, handling schema normalization, violation parsing, DQX validation, and business rule enforcement.

Chicago Transformation

- Violations parsed from a single pipe-delimited Violations column using split and regex.
- Violation code extracted via regex from the numeric prefix of each violation segment.
- Inspection result mapped to violation_score: Pass: 90, Pass w/ Conditions: 80, Fail: 70, No Entry: 0.
- is_critical_flag derived from violation content using keywords “Urgent”/”Critical”.
- restaurant_id generated from DBA_Name, facility_type, address, zip_code.

Dallas Transformation

- Column names normalized (Violation_Description_-_1: violation_description_1).
- 25 wide violation columns unpivoted into one row per violation using a slot-based union pattern (slot_dfs list + unionAll).
- Violation code extracted from the *NN prefix in each violation description field.
- is_critical_flag set to Y where violation_points == 3.
- facility_type derived using regex keyword matching on DBA_Name.
- inspection_id generated from DBA_Name, inspection_date, address, zip_code.

DQX Rules Applied

A custom DQX engine (modeled on Databricks DQX expectations) was implemented with the following checks:

Rule	City	Action
DBA_Name cannot be null or empty	Both	Drop bad row
inspection_date cannot be null	Both	Drop bad row
inspection_type cannot be null or empty	Both	Drop bad row
zip_code must be non-null and match ^\d{5}$	Both	Drop bad row
violation_score must be between 0 and 100	Dallas	Drop bad row
Chicago inspection_result cannot be null	Chicago	Drop bad row
PASS result cannot have any Critical/Urgent violation	Dallas	Drop inspection rows
Score >= 90 cannot have more than 3 violations	Dallas	Drop inspection rows
Duplicate violations deduplicated on (inspection_id, date, violation_code, violation_description)	Both	Keep distinct
Every inspection must have at least 1 unique violation	Both	Validated post-load

DQX Results

- Dallas: 397,106 good rows, 1,300 bad rows (1,186 invalid zip codes, 75 null DBA names, 39 out-of-range scores).
- After business rules and deduplication: 240,211 Dallas rows, 2 Chicago rows written to silver_table.
- Post-write integrity checks all passed: 0 pass+critical violations, 0 scores > 100, 0 Dallas score>=90 with >3 violations, 0 invalid zip codes.

```
============================================================
SILVER LAYER SUMMARY
============================================================
Table : food_inspections.silver.silver_table
Rows  : 1,179,843   Cols: 27

Rows by city:
+-----------+--------------+------------------+
|source_city|violation_rows|unique_inspections|
+-----------+--------------+------------------+
|     DALLAS|        240211|             57738|
|    CHICAGO|        939632|            221740|
+-----------+--------------+------------------+

Score distribution:
+-----------+------------------+---------------+------+
|source_city| inspection_result|violation_score| count|
+-----------+------------------+---------------+------+
|    CHICAGO|         NOT READY|           NULL|   426|
|    CHICAGO|   OUT OF BUSINESS|           NULL|   139|
```

3.4 Silver to Gold (Notebook 04)

Each Gold table is loaded using Delta MERGE (upsert) logic to support idempotent reloads:

dim_date

- Source: DISTINCT inspection_date from silver_table.
- Date attributes computed using Spark date functions (year, month, quarter, weekofyear, date_format).
- MERGE ON date_key: inserts only new dates (WHEN NOT MATCHED).

dim_violation_detail

- Source: DISTINCT (violation_id, violation_code, violation_description, violation_detail) from silver_table.
- MERGE ON violation_id: inserts only new violations (WHEN NOT MATCHED).

dim_restaurant (SCD Type 2)

- Source: DISTINCT (restaurant_id, dba_name, aka_name, facility_type, address, source_city, state, zip_code, change_hash) from silver_table.
- A left anti-join against the current dim_restaurant (is_current = true) is performed first to isolate new or changed records.
- MERGE uses the UNION ALL NULL-key pattern: the source is duplicated: one copy with the actual restaurant_id as merge_key (to trigger updates on matched records), and one with NULL merge_key (to trigger inserts).
- WHEN MATCHED AND change_hash differs: expire old record (is_current = false, effective_end_date = current_timestamp).
- WHEN NOT MATCHED AND merge_key IS NULL: insert new record (effective_start_date = current_timestamp, effective_end_date = 9999-12-31, is_current = true).

fact_inspections

- Source: Silver grouped by (inspection_id, inspection_date, restaurant_id) to compute total_violation (countDistinct violation_id) and total_violation_points (sum violation_points).
- Joined to dim_date to resolve inspection_date_key (FK).
- Joined to dim_restaurant (all versions) to resolve restaurant_key (FK) using restaurant_id + dba_name + aka_name match.
- MERGE ON inspection_id: safe append (WHEN NOT MATCHED only).

fact_violations

- Source: Silver filtered to rows with non-null violation_code.
- Joined to dim_violation_detail to resolve violation_code_key (FK).
- Joined to fact_inspections to resolve inspection_key (FK) using the latest record per inspection_id.
- MERGE ON (inspection_key, violation_code_key): safe append (WHEN NOT MATCHED only), preventing duplicate violations per inspection.

Load Dim Date

07:08 PM (15s) 5 Python

```python
# ── dim_date ──────────────────────────────────

dim_date = (
    silver
    .select("inspection_date")
    .distinct()
    .filter(F.col("inspection_date").isNotNull())
    .withColumn("date_key",           F.date_format("inspection_date", "yyyyMMdd").cast(IntegerType()))
    .withColumn("full_date",          F.col("inspection_date"))
    .withColumn("year",               F.year("inspection_date"))
    .withColumn("month_name",         F.date_format("inspection_date", "MMMM"))
    .withColumn("month_number",       F.month("inspection_date"))
    .withColumn("day",                F.date_format("inspection_date", "EEEE"))
    .withColumn("quarter",            F.concat(F.lit("Q"), F.quarter("inspection_date")))
    .withColumn("week_number",        F.weekofyear("inspection_date"))
    .withColumn("_date_to_warehouse", F.current_timestamp())
    .select(
        "date_key", "full_date", "year", "month_name",
        "month_number", "day", "quarter", "week_number",
        "_date_to_warehouse"
    )
)

dim_date.createOrReplaceTempView("staging_dim_date")

spark.sql(f"""
    MERGE INTO {CATALOG}.{GOLD_SCHEMA}.{DIM_DATE} AS target
    USING staging_dim_date AS source
    ON target.date_key = source.date_key
    WHEN NOT MATCHED THEN INSERT *
""")

# dim_date.write.format("delta").mode("append")\
#     .saveAsTable(f"{CATALOG}.{GOLD_SCHEMA}.{DIM_DATE}")

get_merge_stats(DIM_DATE).display()
```

See performance (3)

dim_date: pyspark.sql.connect.dataframe.DataFrame = [date_key: integer, full_date: date ... 7 more fields]

Table +

	timestamp	job	operati...	numOutput...	numSource...	numTargetRowsI...	Status
1	2026-04-12T23:08:...	null	MERGE	4519	4519	4519	Data Ins...

Load Fact Violations

07:09 PM (10s) 16 Python

Generate (⌘ + I)

```python
dim_viol_lkp = (
    spark.table(f"{CATALOG}.{GOLD_SCHEMA}.{DIM_VIOLATION_DETAIL}")
    .select("violation_code_key", "violation_id")
)

w = Window.partitionBy("inspection_id").orderBy(F.col("inspection_score").desc())

fact_insp_lkp = (
    spark.table(f"{CATALOG}.{GOLD_SCHEMA}.{FACT_INSPECTIONS}")
    .select("inspection_key", "inspection_id", "inspection_score")
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1)
    .drop("rn")
    .select("inspection_key", "inspection_id")
)

fact_violations = (
    silver
    .filter(
        F.col("violation_code").isNotNull() &
        (F.trim(F.col("violation_code")) != ""))

    # Resolve violation_code_key from dim_violation
    .join(dim_viol_lkp,
          on=["violation_id"],
          how="left")
    # Resolve inspection_key from fact_inspections
    .join(fact_insp_lkp,
          on="inspection_id",
          how="left")
    .withColumn("_date_to_warehouse", F.current_timestamp())
    .select(
        "inspection_key",
        "violation_code_key",
        F.col("violation_points"),
        F.col("is_critical_flag").alias("is_critical"),
        F.col("inspector_comments").alias("violation_comment"),
        "_date_to_warehouse"
    )
)

fact_violations.createOrReplaceTempView("staging_fact_violations")
```

Click to add a breakpoint.

```python
    MERGE INTO {CATALOG}.{GOLD_SCHEMA}.{FACT_VIOLATIONS} target
    USING staging_fact_violations source
    ON target.inspection_key = source.inspection_key
    AND target.violation_code_key = source.violation_code_key

    -- Safe Append
    WHEN NOT MATCHED THEN INSERT (
        inspection_key, violation_code_key, violation_points, is_critical, violation_comment, _date_to_warehouse
    )
    Values (
        source.inspection_key, source.violation_code_key, source.violation_points, source.is_critical, source.violation_comment, source._date_to_warehouse
    )
""")

get_merge_stats(FACT_VIOLATIONS).display()
```

See performance (3)

dim_viol_lkp: pyspark.sql.connect.dataframe.DataFrame = [violation_code_key: long, violation_id: string]
fact_insp_lkp: pyspark.sql.connect.dataframe.DataFrame = [inspection_key: long, inspection_id: string]
fact_violations: pyspark.sql.connect.dataframe.DataFrame = [inspection_key: long, violation_code_key: long ... 4 more fields]

Table +

	timestamp	job	operati...	numOutput...	numSource...	numTargetRowsI...	Status
1	2026-04-12T23:09:...	null	MERGE	1179883	1179883	1179883	Data Ins...

3.5 Quarantine Table

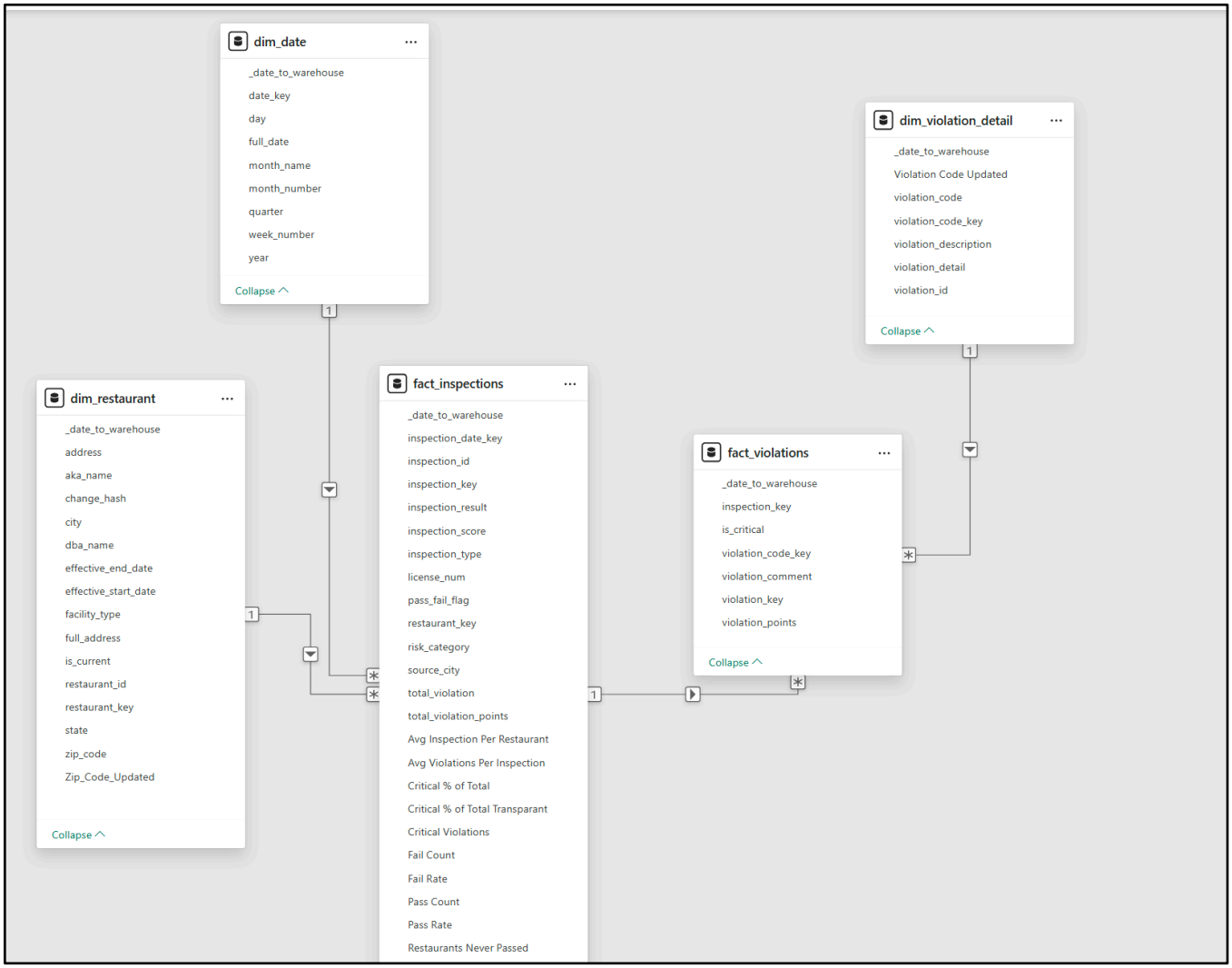
A quarantine table (gold_quarantine_inspection) was implemented to capture any inspection records that would have been duplicated when loading fact_inspections. Records are tagged with quarantine_category, failed_column, source_system, and pipeline_name for audit and investigation purposes.

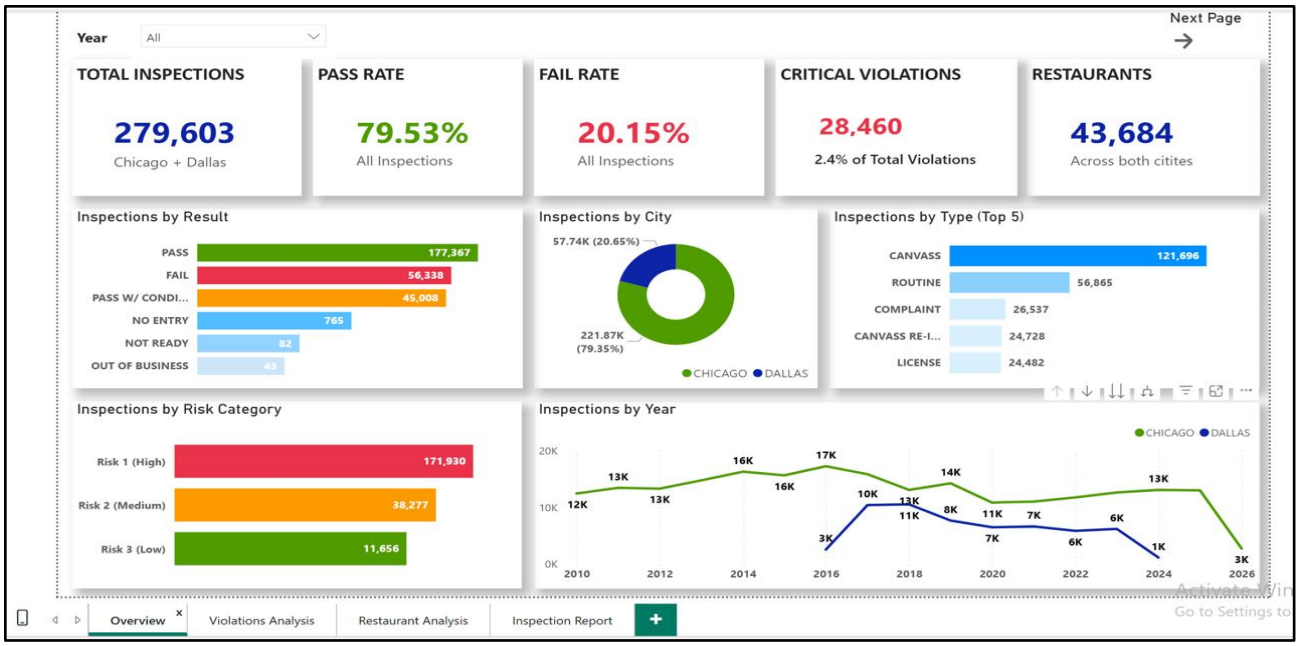
```python
def quarantine_fact_inspection(quarantine_df):
    quarantine_df.createOrReplaceTempView("staging_quarantine_inspection")
    spark.sql(f"""
        INSERT INTO {CATALOG}.{GOLD_SCHEMA}.{GOLD_QUARANTINE_INSPECTION}
        SELECT
            inspection_id,
            restaurant_id,
            restaurant_name,
            aka_name,
            CAST(inspection_date AS DATE),
            inspection_type,
            inspection_result,
            violation_score,
            violation_code,
            violation_points,
            pass_fail_flag,
            source_city,
            address,
            zip_code,
            quarantine_category,
            failed_column,
            source_system,
            pipeline_name,
            current_timestamp() AS _date_to_warehouse,
            current_timestamp()AS quarantine_timestamp
        FROM staging_quarantine_inspection
    """)
```

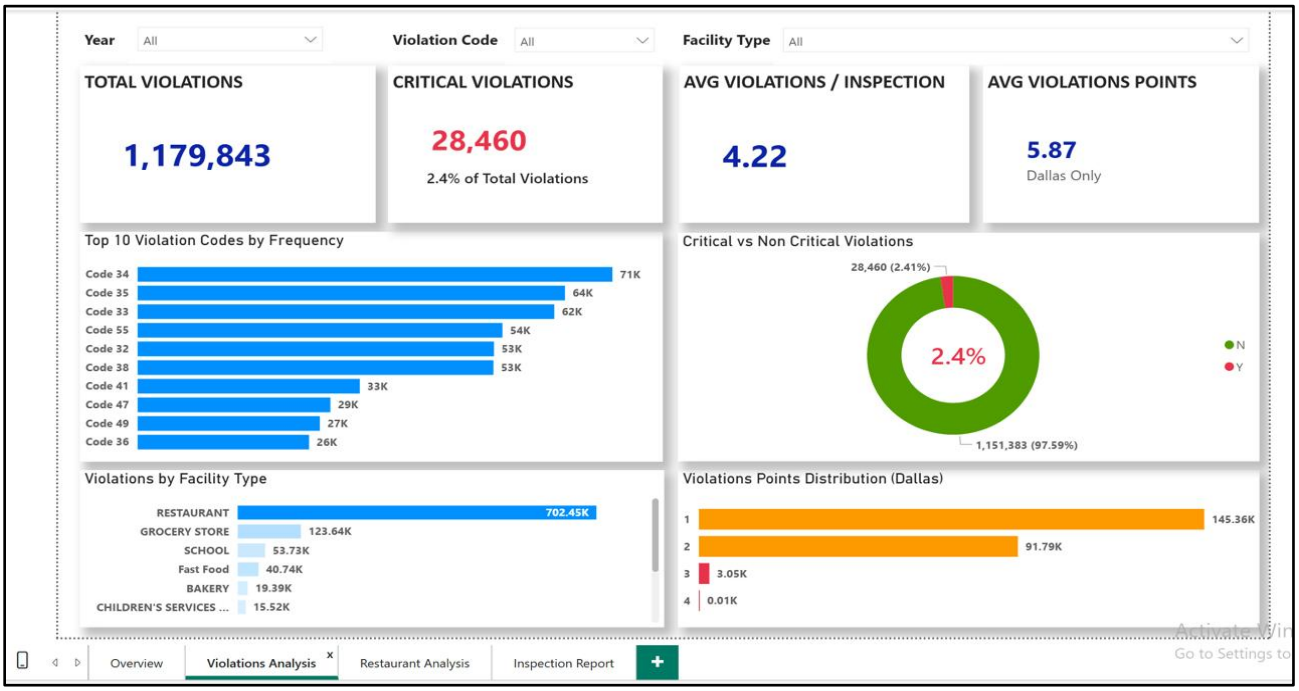
Part 4: BI Dashboards

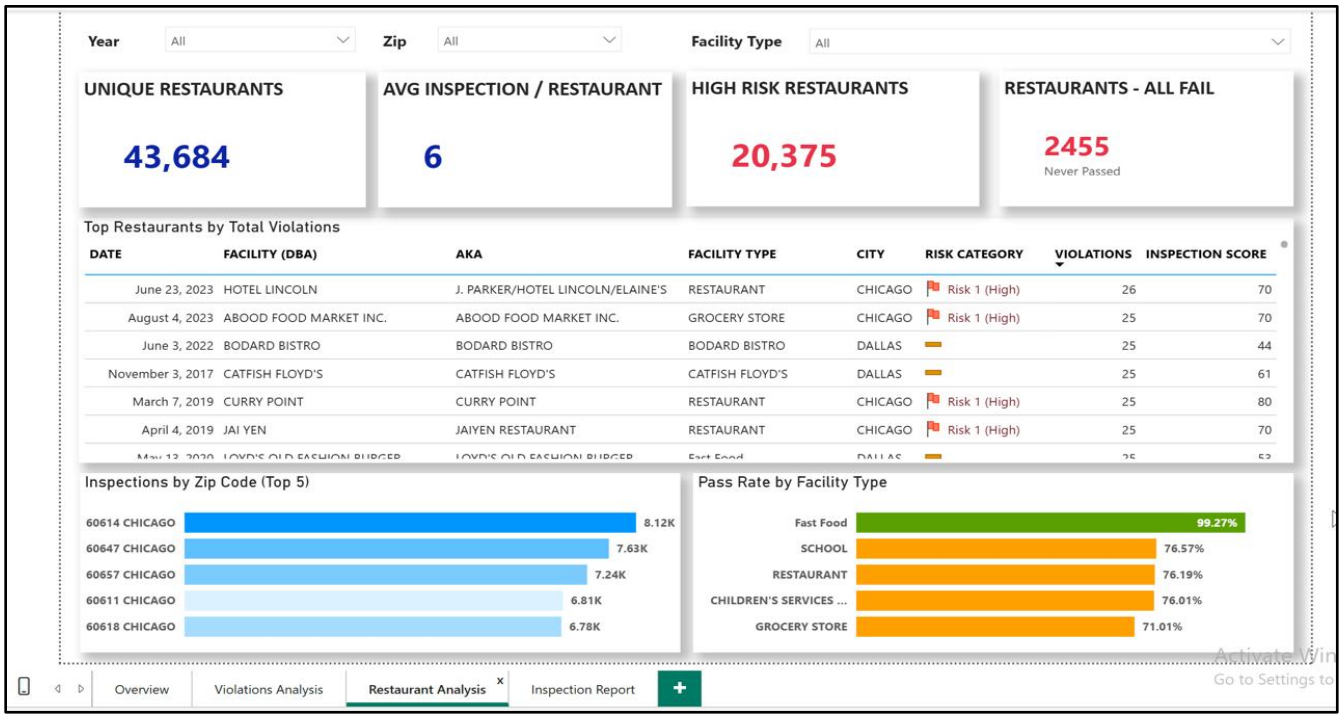
4.1 Dashboard Overview

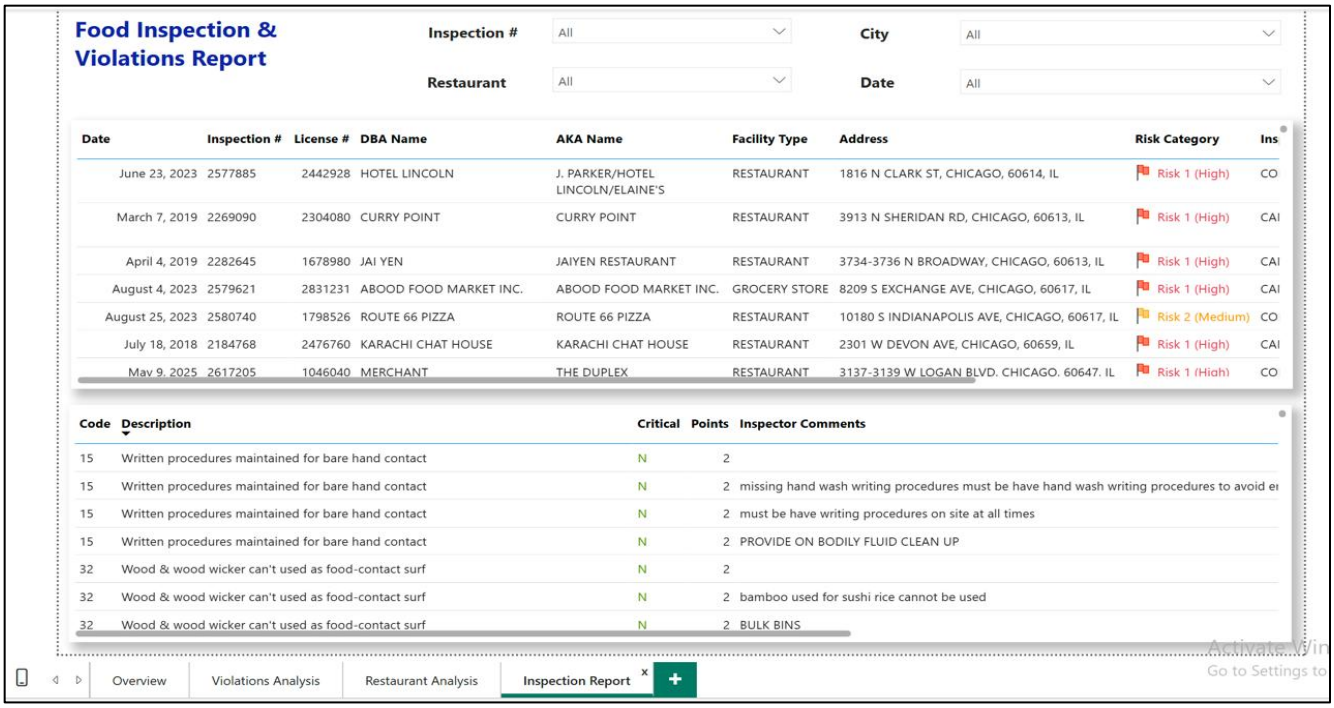
BI dashboards were created using [Power BI to visualize food inspection data from both Chicago and Dallas. The dashboards leverage the Gold Zone dimensional model for all analytics.




Year
All
Next Page
TOTAL INSPECTIONS
279,603
Chicago + Dallas
PASS RATE
79.53%
All Inspections
FAIL RATE
20.15%
All Inspections
CRITICAL VIOLATIONS
28,460
2.4% of Total Violations
RESTAURANTS
43,684
Across both citites
Inspections by Result
PASS 177,367
FAIL 56,338
PASS W/ CONDI... 45,008
NO ENTRY 765
NOT READY 82
OUT OF BUSINESS 43
Inspections by City
57.74K (20.65%)
221.87K (79.35%)
CHICAGO DALLAS
Inspections by Type (Top 5)
CANVASS 121,696
ROUTINE 56,865
COMPLAINT 26,537
CANVASS RE-I... 24,728
LICENSE 24,482
Inspections by Risk Category
Risk 1 (High) 171,930
Risk 2 (Medium) 38,277
Risk 3 (Low) 11,656
Inspections by Year
CHICAGO DALLAS
20K
10K
0K
2010
2012
2014
2016
2018
2020
2022
2024
2026
Overview
Violations Analysis
Restaurant Analysis
Inspection Report


Year
All
Violation Code
All
Facility Type
All
TOTAL VIOLATIONS
1,179,843
CRITICAL VIOLATIONS
28,460
2.4% of Total Violations
AVG VIOLATIONS / INSPECTION
4.22
AVG VIOLATIONS POINTS
5.87
Dallas Only
Top 10 Violation Codes by Frequency
Code 34 71K
Code 35 64K
Code 33 62K
Code 55 54K
Code 32 53K
Code 38 53K
Code 41 33K
Code 47 29K
Code 49 27K
Code 36 26K
Critical vs Non Critical Violations
28,460 (2.41%)
2.4%
1,151,383 (97.59%)
N
Y
Violations by Facility Type
RESTAURANT 702.45K
GROCERY STORE 123.64K
SCHOOL 53.73K
Fast Food 40.74K
BAKERY 19.39K
CHILDREN'S SERVICES ... 15.52K
Violations Points Distribution (Dallas)
1 145.36K
2 91.79K
3 3.05K
4 0.01K
Overview
Violations Analysis
Restaurant Analysis
Inspection Report


Year All
Zip All
Facility Type All
UNIQUE RESTAURANTS
43,684
AVG INSPECTION / RESTAURANT
6
HIGH RISK RESTAURANTS
20,375
RESTAURANTS - ALL FAIL
2455
Never Passed
Top Restaurants by Total Violations
DATE FACILITY (DBA) AKA FACILITY TYPE CITY RISK CATEGORY VIOLATIONS INSPECTION SCORE
June 23, 2023 HOTEL LINCOLN J. PARKER/HOTEL LINCOLN/ELAINE'S RESTAURANT CHICAGO Risk 1 (High) 26 70
August 4, 2023 ABOOD FOOD MARKET INC. ABOOD FOOD MARKET INC. GROCERY STORE CHICAGO Risk 1 (High) 25 70
June 3, 2022 BODARD BISTRO BODARD BISTRO BODARD BISTRO DALLAS 25 44
November 3, 2017 CATFISH FLOYD'S CATFISH FLOYD'S CATFISH FLOYD'S DALLAS 25 61
March 7, 2019 CURRY POINT CURRY POINT RESTAURANT CHICAGO Risk 1 (High) 25 80
April 4, 2019 JAI YEN JAIYEN RESTAURANT RESTAURANT CHICAGO Risk 1 (High) 25 70
Inspections by Zip Code (Top 5)
60614 CHICAGO 8.12K
60647 CHICAGO 7.63K
60657 CHICAGO 7.24K
60611 CHICAGO 6.81K
60618 CHICAGO 6.78K
Pass Rate by Facility Type
Fast Food 99.27%
SCHOOL 76.57%
RESTAURANT 76.19%
CHILDREN'S SERVICES ... 76.01%
GROCERY STORE 71.01%
Overview
Violations Analysis
Restaurant Analysis
Inspection Report


Food Inspection & Violations Report
Inspection # All
City All
Restaurant All
Date All
Date Inspection # License # DBA Name AKA Name Facility Type Address Risk Category
June 23, 2023 2577885 2442928 HOTEL LINCOLN J. PARKER/HOTEL LINCOLN/ELAINE'S RESTAURANT 1816 N CLARK ST, CHICAGO, 60614, IL Risk 1 (High)
March 7, 2019 2269090 2304080 CURRY POINT CURRY POINT RESTAURANT 3913 N SHERIDAN RD, CHICAGO, 60613, IL Risk 1 (High)
April 4, 2019 2282645 1678980 JAI YEN JAIYEN RESTAURANT RESTAURANT 3734-3736 N BROADWAY, CHICAGO, 60613, IL Risk 1 (High)
August 4, 2023 2579621 2831231 ABOOD FOOD MARKET INC. ABOOD FOOD MARKET INC. GROCERY STORE 8209 S EXCHANGE AVE, CHICAGO, 60617, IL Risk 1 (High)
August 25, 2023 2580740 1798526 ROUTE 66 PIZZA ROUTE 66 PIZZA RESTAURANT 10180 S INDIANAPOLIS AVE, CHICAGO, 60617, IL Risk 2 (Medium)
July 18, 2018 2184768 2476760 KARACHI CHAT HOUSE KARACHI CHAT HOUSE RESTAURANT 2301 W DEVON AVE, CHICAGO, 60659, IL Risk 1 (High)
May 9, 2025 2617205 1046040 MERCHANT THE DUPLEX RESTAURANT 3137-3139 W LOGAN BLVD, CHICAGO, 60647, IL Risk 1 (High)
Code Description Critical Points Inspector Comments
15 Written procedures maintained for bare hand contact N 2
15 Written procedures maintained for bare hand contact N 2 missing hand wash writing procedures must be have hand wash writing procedures to avoid er
15 Written procedures maintained for bare hand contact N 2 must be have writing procedures on site at all times
15 Written procedures maintained for bare hand contact N 2 PROVIDE ON BODILY FLUID CLEAN UP
32 Wood & wood wicker can't used as food-contact surf N 2
32 Wood & wood wicker can't used as food-contact surf N 2 bamboo used for sushi rice cannot be used
32 Wood & wood wicker can't used as food-contact surf N 2 BULK BINS
Overview
Violations Analysis
Restaurant Analysis
Inspection Report

Good Practices Incorporated

Practice	Implementation
Medallion Architecture	Raw Volume → Staging Volume (Parquet) → Bronze (Delta) → Silver (Delta) → Gold (Delta)
Metadata-Driven Pipeline	Parent metadata table drives active source selection: child metrics table logs run status, row counts, and batch IDs
Idempotent Loads	All Gold loads use Delta MERGE (not overwrite), ensuring reruns are safe and do not duplicate data
Data Quality (DQX)	Custom DQX engine applies expectations at Silver layer; bad rows tracked with error message column
SCD Type 2	dim_restaurant implements full Type 2 with change_hash detection, effective dates, and is_current flag
MD5 Surrogate Keys	Consistent business key generation via MD5 hashing for inspection_id, restaurant_id, and violation_id ensures cross-city compatibility
Column Standardization	All column names lowercased, spaces replaced with underscores, special characters removed before Silver load
Audit Columns	_date_to_warehouse, _batch_id, _ingested_at, _bronze_loaded_at added at each layer for lineage tracking
Quarantine Pattern	Duplicate or invalid Gold-layer records routed to a quarantine table with reason codes
Git Integration	All notebooks committed to Git; post-due-date commits excluded per course requirements